



UNIDAD 2

Recuperación avanzada de bases de datos.

TEMA 1:

Fundamentos de recuperación.

Descripción:

La **recuperación** avanzada de bases de datos constituye un pilar fundamental en la administración de la información, al garantizar la **integridad**, **consistencia** y **disponibilidad** de los datos frente a fallos o desastres, en esta unidad se abordan los conceptos teóricos y prácticos relacionados con **estrategias** de consistencia, control de concurrencia, respaldos, restauración y métodos de recuperación en distintos sistemas gestores.

Asimismo, se analizan políticas y buenas prácticas que permiten diseñar planes eficientes de continuidad del negocio, para fortalecer las competencias técnicas de los estudiantes en escenarios reales de gestión y recuperación de datos en DBMS.

Administración de bases de datos.

PERIODO ACADÉMICO 2025-2S

TABLA DE CONTENIDO

Tema 1. Fundamentos de recuperación.	2
1.1. Estrategias de consistencia.....	2
1.2. Control de concurrencia.	6
1.3. Copias de seguridad y tipos de respaldo.	11
1.4. Planificación de respaldos y estrategias de recuperación.	14
Bibliografía	21

Tema 1. Fundamentos de recuperación.

1.1. Estrategias de consistencia.

La consistencia en una base de datos garantiza que, después de ejecutar una transacción, los datos se mantengan correctos y cumplan con las reglas definidas en el modelo y en las restricciones de integridad, las estrategias de consistencia se centran en aplicar controles que eviten anomalías, como registros duplicados o violaciones de claves foráneas, y en asegurar que toda transacción respete las propiedades ACID.

De este modo, al finalizar cada operación, la base de datos pasa de un estado válido a otro igualmente válido, preserva la confiabilidad de la información almacenada (Silberschatz et al., 2020a, Armbrust et al., 2020).

La consistencia es una de las propiedades esenciales de las transacciones dentro del modelo **ACID**, esta asegura que cualquier transacción lleva a la base de datos de un estado válido a otro también válido, respeta las restricciones de integridad (claves primarias, foráneas, reglas de negocio) (Silberschatz et al., 2020).

Las estrategias de consistencia implican:

- **Establecer reglas claras de integridad** cuando se diseña el esquema.
- **Aplicar validaciones de forma automática**, en el SGBD utilizando constraints y triggers.
- **Diseñar transacciones que cumplan con atomicidad y aislamiento**, para evitar anomalías como actualizaciones perdidas o violaciones de integridad.

Estrategias

Control

Copias

Planificación

La consistencia es el puente que conecta la lógica de negocio con la confiabilidad técnica de la base de datos (Elmasri & Navathe, 2020).

Ejemplo:

Código SQL

```
-- Creación de una tabla con restricciones de integridad
CREATE TABLE Empleados (
  ID INT PRIMARY KEY,
  Nombre NVARCHAR(50) NOT NULL,
  Salario DECIMAL(10,2) CHECK (Salario > 0),
  DepartamentoID INT FOREIGN KEY REFERENCES Departamentos(ID)
);

-- Transacción que respeta la consistencia
BEGIN TRANSACTION;
  INSERT INTO Empleados (ID, Nombre, Salario, DepartamentoID)
  VALUES (1, 'Ana Torres', 1200, 10);
COMMIT;
```

Si se intenta insertar un salario negativo o un departamento inexistente, la transacción falla y la consistencia se mantiene.

La consistencia es una de las propiedades esenciales dentro del modelo **ACID**, que rige el comportamiento de las transacciones en bases de datos, estas propiedades son: **Atomicidad**, que asegura que una transacción se ejecuta en su totalidad o no se ejecuta; **Consistencia**, que garantiza que los cambios respeten todas las reglas de integridad, lleva a la base de datos de un estado válido a otro válido; **Aislamiento**, que permite que las transacciones concurrentes se ejecuten sin interferir entre sí, como si se ejecutaran en serie; y **Durabilidad**, que asegura que una vez confirmada la transacción, sus efectos persisten incluso ante fallos del sistema. Así, las estrategias de consistencia no se entienden de manera aislada, sino como parte de un marco integral de control transaccional (Silberschatz et al., 2020).

Estrategias

Control

Copias

Planificación

 Ejemplo:

Atomicidad: Si la transacción no es atómica, una parte se ejecuta y otra no, deja la base en un estado inconsistente.

Código SQL

```
BEGIN TRANSACTION;
  INSERT INTO Empleados (ID, Nombre, Salario, DepartamentoID)
  VALUES (2, 'María López', 1200, 5);

  -- Error: el departamento 99 no existe
  INSERT INTO Empleados (ID, Nombre, Salario, DepartamentoID)
  VALUES (3, 'Pedro Ruiz', 1000, 99);
COMMIT;
```

Sin **atomicidad**, el primer INSERT quedaría guardado, aunque el segundo falle, con atomicidad, **todo se revierte** y la base queda igual que antes de la transacción.

 Ejemplo:

Consistencia: Si no hay consistencia, las restricciones de integridad no se respetan.

Código SQL

```
INSERT INTO empleados (nombre, salario, departamento_id)
VALUES ('Luis Gómez', -500, 1); -- Salario negativo
```

La **consistencia** evita este registro gracias al CHECK (salario > 0), si la consistencia no existiera, el sistema aceptaría valores inválidos y los datos perderían confiabilidad.

Estrategias

Control

Copias

Planificación

● Ejemplo:

Aislamiento: Sin aislamiento, las transacciones concurrentes se afectan entre sí.

Código SQL

```
-- Transacción A
BEGIN TRANSACTION;
UPDATE Cuentas SET saldo = saldo - 100 WHERE id = 1;

-- Transacción B (ejecutada al mismo tiempo)
BEGIN TRANSACTION;
UPDATE Cuentas SET saldo = saldo - 200 WHERE id = 1;
```

Si no hay aislamiento, ambas transacciones podrían leer el mismo saldo inicial, **provoca un sobregiro no detectado**, con aislamiento, el sistema asegura que cada transacción vea datos consistentes.

● Ejemplo:

Durabilidad: Si no se toma en cuenta la durabilidad, luego de confirmada la transacción, esta podría perderse ante un cualquier fallo.

Código SQL

```
BEGIN;
INSERT INTO empleados (nombre, salario, departamento_id)
VALUES ('Sofía Herrera', 1800, 2);
COMMIT;
```

Sin durabilidad, un corte de energía posterior al COMMIT borraría el registro, con durabilidad, los cambios quedan escritos en los **logs (WAL en PostgreSQL, Transaction Log en SQL Server)** y se recuperan automáticamente.

1.2. Control de concurrencia.

El control de concurrencia es el conjunto de técnicas y mecanismos que utilizan los sistemas de bases de datos para garantizar que múltiples transacciones que se ejecutan de manera simultánea no generen inconsistencias, en el caso que varios usuarios acceden y modifican datos al mismo tiempo, pueden producirse problemas como actualizaciones perdidas, lecturas sucias o resultados no repetibles.

El objetivo del control de concurrencia es preservar la aislación de las transacciones (propiedad ACID), de modo que el resultado final sea el mismo que si las transacciones se hubieran ejecutado en serie (Zhao et al., 2022).

Entre los métodos de control de concurrencia se encuentran:

- **Bloqueo (Locking):** Se emplean bloqueos compartidos en lectura y bloqueos exclusivos para escritura, para evitar que dos transacciones modifiquen el mismo dato al mismo tiempo (Silberschatz et al., 2020).
- **Control por marcas de tiempo (Timestamp Ordering):** Cada transacción recibe una marca de tiempo y el sistema asegura que las operaciones se ejecuten en orden cronológico lógico (Silberschatz et al., 2020).
- **Control multiversión (MVCC):** Mantiene varias versiones de un dato para que las transacciones lean un estado consistente mientras otras escriben (Silberschatz et al., 2020).
- **Detección y prevención de deadlocks:** El sistema debe identificar o evitar situaciones en las que varias transacciones esperen indefinidamente por recursos bloqueados entre sí (Silberschatz et al., 2020).

La concurrencia busca una relación óptima entre **consistencia** y **eficiencia**, para evitar conflictos mientras se mantiene un alto nivel de rendimiento en el acceso simultáneo a los datos (Silberschatz et al., 2020).

● Ejemplo:

Bloqueo (Locking): Para evitar que un registro se modifique al mismo tiempo por dos transacciones se usa el bloqueo.

Código SQL

```
-- Sesión 1
BEGIN TRANSACTION;
UPDATE Cuentas SET saldo = saldo + 100 WHERE id = 1;
-- Esta transacción mantiene un bloqueo EXCLUSIVO sobre id=1

-- Sesión 2 (ejecutada en paralelo)
BEGIN TRANSACTION;
UPDATE Cuentas SET saldo = saldo - 50 WHERE id = 1;
-- Se queda y espera hasta que la Sesión 1 haga COMMIT o ROLLBACK
```

El bloqueo asegura que el dato no se corrompa: solo una transacción puede escribir a la vez.

● Ejemplo:

Control por marcas de tiempo (Timestamp Ordering): Aunque no siempre se puede “forzar” directamente, se puede ilustrarlo con un ejemplo conceptual.

Cada transacción recibe un *timestamp* al comenzar, el SGBD ordena las operaciones de acuerdo con esos sellos temporales, evita conflictos

Código SQL

```
-- Transacción T1 (Timestamp = 10)
BEGIN;
UPDATE Productos SET stock = stock - 5 WHERE id = 1;
COMMIT;

-- Transacción T2 (Timestamp = 12)
BEGIN;
UPDATE Productos SET stock = stock - 3 WHERE id = 1;
COMMIT;
```

Estrategias

Control

Copias

Planificación

El sistema respeta el orden de timestamps (T1 antes que T2), si T2 intentara leer un valor “viejo” modificado por T1, el sistema rechaza o reinicia la transacción para asegurar serialización.

● Ejemplo:

MVCC: Mantiene varias versiones de un dato para que las transacciones lean un estado

Código SQL

```
-- Sesión 1
BEGIN;
UPDATE empleados SET salario = salario + 100 WHERE id = 1;

-- Sesión 2 (antes de que Sesión 1 haga COMMIT)
SELECT * FROM empleados WHERE id = 1;
```

Gracias a **MVCC**, la **Sesión 2** ve la versión anterior del registro, sin el aumento de salario, hasta que la **Sesión 1 confirme el COMMIT**, esto asegura que cada transacción lea datos consistentes sin bloquear excesivamente el sistema.

Anomalías clásicas

Lectura sucia (Dirty Read): Cuando una transacción lee datos que otra no ha confirmado, si la segunda hace ROLLBACK y termina, la primera habrá usado información no válida.

Estrategias

Control

Copias

Planificación

 Ejemplo:

Código SQL

```
-- Sesión 1
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
BEGIN TRANSACTION;
SELECT saldo FROM Cuentas WHERE id = 1; --Puede leer un valor aún no
confirmado

-- Sesión 2
BEGIN TRANSACTION;
UPDATE Cuentas SET saldo = saldo - 500 WHERE id = 1;
-- No se ha hecho COMMIT
ROLLBACK; -- El cambio nunca fue real, pero Sesión 1 lo "vio"
```

El control de concurrencia con aislamiento **READ COMMITTED** o superior evita esta anomalía

Lectura no repetible (Non-repeatable Read): Una transacción lee el mismo dato dos veces y obtiene resultados distintos porque otra transacción lo modificó en medio

 Ejemplo:

Código SQL

```
-- Sesión 1
BEGIN;
SELECT salario FROM empleados WHERE id = 1; -- Resultado: 1000

-- Sesión 2
BEGIN;
UPDATE empleados SET salario = 1200 WHERE id = 1;
COMMIT;

-- Sesión 1 (segunda lectura)
SELECT salario FROM empleados WHERE id = 1; -- Resultado: 1200
(cambió durante la transacción)
COMMIT;
```

Se evita con aislamiento **REPEATABLE READ** o superior.

Estrategias

Control

Copias

Planificación

Lectura fantasma (Phantom Read): Una transacción vuelve a ejecutar una consulta y aparecen **nuevos registros** que antes no existían porque otra transacción los insertó

● Ejemplo:

Código SQL

```
-- Sesión 1
BEGIN TRANSACTION;
SELECT * FROM Pedidos WHERE monto > 1000; -- Devuelve 5 filas

-- Sesión 2
INSERT INTO Pedidos (id, monto) VALUES (101, 1500);
COMMIT;

-- Sesión 1 (segunda lectura)
SELECT * FROM Pedidos WHERE monto > 1000; -- Ahora devuelve 6 filas
(un "fantasma")
COMMIT;
```

Se evita con aislamiento **SERIALIZABLE**, que bloquea rangos de datos y no solo filas individuales.

1.3. Copias de seguridad y tipos de respaldo.

Copias de seguridad.

Las copias de seguridad constituyen la primera línea de defensa frente a fallos, pérdidas de información o desastres, una copia de seguridad consiste en duplicar total o parcialmente los datos almacenados en la base de datos para garantizar su disponibilidad en caso de error o daño, su propósito no es solo proteger la información, sino también asegurar la continuidad de las operaciones en las organizaciones, un plan de respaldo bien diseñado permite reducir riesgos, optimizar tiempos de recuperación y ofrecer mayor confianza en la gestión de los datos (Thallapally, 2025).

Tipos de copias de seguridad.

Elegir una combinación eficaz de diferentes tipos de copias de seguridad puede afectar considerablemente el coste y la eficacia de su estrategia de protección de datos, ya que cada tipo de copia de seguridad tiene sus propias ventajas y desventajas (Thallapally, 2025). Los tres tipos principales de copias de seguridad son:

- **Copia completa (Full Backup):** Consiste en duplicar la totalidad de los datos de la base de datos, son fáciles de implementar y permiten restaurar el sistema en una sola operación. Sin embargo, requieren mayor tiempo de ejecución y un espacio de almacenamiento considerable.
- **Copia incremental (Incremental Backup):** Almacena solo los cambios realizados desde el último respaldo (sea completo o incremental), esto optimiza el uso de espacio y agiliza la ejecución del respaldo. El inconveniente es que la recuperación es más lenta, pues se necesita el último respaldo completo y todos los incrementales posteriores.

- **Copia diferencial (Differential Backup):** Registra todos los cambios efectuados desde el último respaldo completo, ocupa más espacio que un respaldo incremental, su restauración es más rápida porque solo se necesita el último respaldo completo y el diferencial más reciente.

Copias Físicas y Lógicas

- **Copias físicas:** Son copias que se realiza directamente de los todos los archivos binarios de la DB, como los ficheros de datos, logs de transacciones y metadatos, este método es eficiente y rápido, pero suele depender de herramientas específicas del sistema gestor.

● Ejemplo:

En PostgreSQL, usar `pg_basebackup`; en SQL Server, copiar archivos `.mdf` y `.ldf` dentro de un plan de respaldo.

- **Copias lógicas:** Trabaja con las estructuras exportándolas como: tablas, esquemas, procedimientos y los datos en un formato legible, como scripts SQL, como ventaja permiten migrar datos entre versiones o sistemas distintos, aunque la restauración puede tardar más.

● Ejemplo:

En PostgreSQL, `pg_dump` y `pg_dumpall`; en SQL Server, el asistente de exportación de datos.

Estrategias

Control

Copias

Planificación

Código SQL

```
-- Copia completa
BACKUP DATABASE Empresa
TO DISK = 'C:\Respaldos\Empresa_Full.bak';

-- Copia diferencial
BACKUP DATABASE Empresa
TO DISK = 'C:\Respaldos\Empresa_Diff.bak'
WITH DIFFERENTIAL;
```

Tabla comparativa de los tipos de respaldo (Thallapally, 2025).

Tipo de respaldo	Ventajas	Desventajas	¿Cuándo usarlo?
Completo (Full Backup)	- Copia íntegra de la base de datos. - Restauración rápida y directa. - Sencillo de implementar.	- Alto consumo de espacio. - Tiempo prolongado para grandes bases. - Poco eficiente si se ejecuta con frecuencia.	- Inicio de cualquier plan de respaldo. - Antes de migraciones o actualizaciones importantes. - Sistemas críticos donde la restauración rápida es prioritaria.
Diferencial (Differential Backup)	- Más rápido que el completo. - Restauración simple (último full + último diferencial). - Balance entre espacio y tiempo.	- Ocupa más espacio que los incrementales. - A medida que pasa el tiempo, el tamaño del diferencial crece.	- Respaldo diario en combinación con un full semanal. - Se requiere restauración rápida sin tanto almacenamiento extra.
Incremental Backup	- Ahorro de espacio. - Respaldo muy rápido. - Permite mayor frecuencia de copias.	- Restauración más lenta (requiere el full + todos los incrementales). - Complejo de administrar.	- Grandes bases con cambios frecuentes. - El tiempo de ejecución del respaldo es crítico. - Escenarios con almacenamiento limitado.

Estrategias

Control

Copias

Planificación

Copias físicas

- Muy rápidas.
- Copia directa de archivos binarios.
- Preservan configuraciones del motor.

- Dependientes del motor y la versión.
- No portables entre diferentes gestores.

- Recuperación ante fallos del servidor.
- Migraciones dentro del mismo motor o versión compatible.

Copias lógicas

- Portables entre diferentes motores o versiones.
- Se pueden editar, revisar o filtrar datos.
- Útiles para migraciones.

- Restauración más lenta.
- Archivos más pesados.
- No siempre conserva configuraciones avanzadas.

- Migración entre gestores distintos.
- Exportación selectiva de tablas.
- Respallos para auditoría o análisis externo.

1.4. Planificación de respaldos y estrategias de recuperación.

La planificación de respaldos y estrategias de recuperación son procesos organizacionales que define: **qué respaldar, cuándo hacerlo y cómo recuperarlo** en caso de fallo; generar un respaldo no solo es ejecutar copias de seguridad; es necesario establecer políticas claras que consideren aspectos como la frecuencia de respaldo, los recursos disponibles y los objetivos de la organización (Elmasri & Navathe, 2020).

Una planificación adecuada garantiza que los datos puedan recuperarse en el menor tiempo posible, reducir riesgos y tiempos de inactividad, también permite definir métricas como el **RPO (Recovery Point Objective)** y el **RTO (Recovery Time Objective)**, que orientan las decisiones técnicas y estratégicas (Zgureanu, 2022).

Elementos de un plan de respaldos y recuperación.

El plan de recuperación constituye una herramienta primordial de las organizaciones tanto públicas como privadas, ya que garantizará la continuidad de las operaciones. Como elementos principales se consideran los siguientes:

Estrategias

Control

Copias

Planificación

- **Frecuencia de respaldos:** Define cada cuánto se deben realizar las copias de seguridad, puede variar desde minutos (en sistemas financieros críticos) hasta diarios o semanales en sistemas menos exigentes. Un buen diseño combina respaldos completos periódicos con diferenciales o incrementales más frecuentes (Silberschatz et al., 2020).
- **Almacenamiento y retención:** Indica dónde se guardarán las copias (servidores locales, nube, cintas magnéticas, discos externos) y por cuánto tiempo se conservarán, esto depende de los recursos disponibles y de normativas legales que exijan mantener la información durante cierto periodo (Silberschatz et al., 2020).
- **RPO (Recovery Point Objective):** Es la tolerancia máxima de pérdida de datos. Por ejemplo, si el RPO es de 4 horas, significa que se pueden perder como máximo las últimas 4 horas de trabajo, por lo que debe haber un respaldo al menos con esa periodicidad (Zgureanu, 2022).
- **RTO (Recovery Time Objective):** Representa el tiempo máximo permitido para recuperar el sistema tras una falla. Un RTO de 2 horas significa que los respaldos y procesos de recuperación deben estar diseñados para restablecer la operación en menos de ese tiempo (Zgureanu, 2022).
- **Pruebas de recuperación:** No basta con generar copias; se debe comprobar periódicamente que las restauraciones funcionan correctamente, muchas organizaciones descubren que sus respaldos son inútiles porque nunca probaron el proceso de recuperación (Silberschatz et al., 2020).
- **Automatización y monitoreo:** Los SGBD modernos y herramientas externas permiten programar respaldos automáticos y generar alertas en caso de fallos. Automatizar evita olvidos humanos y monitorear asegura que siempre se cumpla el plan definido (Silberschatz et al., 2020).

Estrategias

Control

Copias

Planificación

 Ejemplo:

La empresa FinanzasExpress gestiona una base de datos de clientes y transacciones financieras, un fallo en el sistema podría afectar seriamente la confianza de los usuarios, por lo que se diseña un plan de respaldos y recuperación.

Frecuencia de respaldos:

- a. Respaldo completo cada domingo a las 2:00 a.m.
- b. Respaldo diferencial cada noche.
- c. Respaldo incremental (logs de transacciones) cada 30 minutos durante la jornada laboral.

Almacenamiento y retención:

- d. Copias locales en un servidor de respaldo.
- e. Copias en la nube para contingencias mayores.
- f. Retención: 3 meses en almacenamiento local y 1 año en la nube por normativa legal.

RPO (Recovery Point Objective):

- g. 30 minutos → se aceptaría perder, como máximo, la información de la última media hora de operaciones.

RTO (Recovery Time Objective):

- h. 2 horas → el servicio debe estar disponible nuevamente en menos de 120 minutos después de un fallo.

Pruebas de recuperación:

- i. Una vez al mes se ejecuta un simulacro de restauración en un entorno de prueba para validar que los respaldos funcionan correctamente.

Automatización y monitoreo:

- j. En **SQL Server**, se utilizan tareas programadas con *SQL Server Agent*.

Estrategias

Control

Copias

Planificación

- k. En **PostgreSQL**, se programan cron jobs en Linux con alertas por correo en caso de error.

Los sistemas están expuestos a diferentes situaciones de fallo, pero el plan debe asegurar que, aunque ocurra un fallo total, la empresa pueda **restaurar la base de datos en menos de 2 horas** y con una pérdida máxima de datos de 30 minutos, garantiza la continuidad del negocio y la confianza de los clientes.

● Ejemplo:

Automatización con SQL Server Agent
SCRIPT:

Job de respaldo completo semanal con SQL Server Agent
OBJETIVO:

Hacer backup FULL de la BD 'Empresa' cada domingo 2:00 AM
NOTAS:

- Ejecutar con una cuenta con permisos en msdb (sysadmin o SQLAgentOperatorRole).
- Verifica que el servicio "SQL Server Agent" esté iniciado.
- Asegúrate de que la carpeta de destino exista

Código SQL Parte 1

```
/* -----  
0) PARÁMETROS (ajusta según tu entorno)  
----- */  
DECLARE  
    @DBName      sysname      =  
N'Empresa',          -- Nombre de la base a respaldar  
    @BackupPath  nvarchar(260) =  
N'C:\RespalDOS\Empresa_Full.bak', -- Ruta completa del archivo .bak  
    @JobName      sysname      = N'Respaldo_Completo_Empresa',  
    @StepName     sysname      = N'Ejecutar_Backup_Full',  
    @ScheduleName sysname      = N'Horario_Semanal_Domingo_2AM',  
    @Description  nvarchar(4000) = N'Job para ejecutar un respaldo  
completo semanal de la BD Empresa';  
  
/* -----  
1) LIMPIEZA: Si ya existe un Job con el mismo nombre, eliminarlo  
----- */  
IF EXISTS (SELECT 1 FROM msdb.dbo.sysjobs WHERE name = @JobName)  
BEGIN  
    EXEC msdb.dbo.sp_delete_job @job_name = @JobName;  
END  
GO  
  
/* -----  
2) CREAR EL JOB  
----- */  
EXEC msdb.dbo.sp_add_job  
    @job_name      = N'Respaldo_Completo_Empresa',  
    @enabled       = 1,  
    @description= @Description;  
GO
```

Código SQL Parte 2

```
/* -----  
3) AGREGAR PASO (BACKUP FULL)  
- WITH INIT sobrescribe el archivo .bak; quítalo si deseas  
acumular historiales  
----- */  
DECLARE  
    @CmdBackup nvarchar(max) =  
N'BACKUP DATABASE ' + QUOTENAME(N'Empresa') + N'  
TO DISK = ''' + N'C:\Respaldos\Empresa_Full.bak' + N'''  
WITH INIT,  
    NAME = 'Respaldo completo Empresa',  
    CHECKSUM, STATS = 10;';  
  
EXEC msdb.dbo.sp_add_jobstep  
    @job_name      = N'Respaldo_Completo_Empresa',  
    @step_name     = N'Ejecutar_Backup_Full',  
    @subsystem     = N'TSQL',  
    @database_name = N'master',           -- El BACKUP no  
requiere contexto en la BD usuario  
    @command       = @CmdBackup,  
    @on_success_action = 1,               -- Quit  
    @on_fail_action  = 2;                 -- Quit with failure  
  
GO  
  
/* -----  
4) CREAR HORARIO (domingo 2:00 AM)  
- @freq_type = 8 => semanal  
- @freq_interval = 1 => domingo (bitmask: 1=Dom, 2=Lun, 4=Mar,  
8=Mié, 16=Jue, 32=Vie, 64=Sáb)  
- @active_start_time en formato HHMMSS  
----- */  
  
EXEC msdb.dbo.sp_add_schedule  
    @schedule_name = N'Horario_Semanal_Domingo_2AM',  
    @freq_type     = 8,  
    @freq_interval = 1,  
    @active_start_time= 020000;  
  
GO  
  
/* -----  
5) ASOCIAR EL HORARIO AL JOB  
----- */  
  
EXEC msdb.dbo.sp_attach_schedule  
    @job_name      = N'Respaldo_Completo_Empresa',  
    @schedule_name = N'Horario_Semanal_Domingo_2AM';  
  
GO
```

Código SQL Parte 3

```
/* -----  
6) ASIGNAR EL JOB AL SERVIDOR ACTUAL  
----- */  
EXEC msdb.dbo.sp_add_jobserver  
    @job_name = N'Respaldo_Completo_Empresa',  
    @server_name= @@SERVERNAME;  
GO  
  
/* -----  
7) PRUEBA: Ejecutar el Job manualmente (opcional)  
- Útil para validar permisos, ruta y creación del .bak  
----- */  
EXEC msdb.dbo.sp_start_job @job_name = N'Respaldo_Completo_Empresa';  
GO  
  
/* -----  
8) VERIFICAR RESULTADOS (historial del job)  
- Revisa también en SSMS: SQL Server Agent > Jobs > Job History  
----- */  
SELECT TOP (20)  
    j.name AS JobName,  
    h.step_id,  
    h.step_name,  
    h.run_date,  
    h.run_time,  
    h.run_duration,  
    h.sql_message_id,  
    h.sql_severity,  
    h.message  
FROM msdb.dbo.sysjobhistory AS h  
JOIN msdb.dbo.sysjobs AS j ON h.job_id = j.job_id  
WHERE j.name = N'Respaldo_Completo_Empresa'  
ORDER BY h.instance_id DESC;  
GO
```

Bibliografía

Ahmad, M. S., & Panda, B. N. (2024). ClusteredLog: Optimizing Log Structures for Efficient Data Recovery and Integrity Management in Database Systems. *Electronics*, 13(23), 4723. <https://doi.org/10.3390/electronics13234723>

Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., Torres, J., Van Hovell, H., Ionescu, A., Łuszczak, A., Świtakowski, M., Szafranski, M., Li, X., Ueshin, T., Mokhtar, M., Boncz, P., Ghodsi, A., Paranjpye, S., Senster, P., ... Zaharia, M. (2020). Delta lake: High-performance ACID table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 13(12), 3411-3424. <https://doi.org/10.14778/3415478.3415560>

Choi, H., & Lee, S. (2023). Forensic analysis of SQL server transaction log in unallocated area of file system. *Forensic Science International: Digital Investigation*, 46, 301605. <https://doi.org/10.1016/j.fsidi.2023.301605>

Elmasri, R., & Navathe, S. (2020). *Fundamentos de sistemas de bases de datos* (5a ed). Pearson Addison Wesley.

Hayat, Z., & Soomro, T. R. (2018). Implementation of Microsoft SQL Server using 'AlwaysOn' for High Availability and Disaster Recovery without Shared Storage. *International Journal of Experiential Learning & Case Studies*, 3(1), 09-17. <https://doi.org/10.22555/ijelcs.v3i1.1891>

Jorge Santiago Nolasco Valenzuela. (2018). *Python Aplicaciones prácticas*. RA-MA Editorial.

Kassema, J. J. (2016). Disaster Recovery Plan for Business Continuity: Case Study in a Business Sector. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.2796601>

Olasehinde, T., & Wills, F. (2024). DISASTER RECOVERY AND BACKUP STRATEGIES FOR FINANCIAL DATA LAKES. *ResearchGate*. https://www.researchgate.net/publication/387031051_DISASTER_RECOVERY_AND_BACKUP_STRATEGIES_FOR_FINANCIAL_DATA_LAKES

Rodríguez Guerrero, R., Vanegas, C. A., & Castang Montiel, G. (2020). *Python a su alcance*. Editorial Universidad Distrital Francisco José de Caldas. Editorial UD.

Russo, N., Mamede, H. S., Reis, L., Martins, J., & Branco, F. (2023). Exploring a Multidisciplinary Assessment of Organisational Maturity in Business

Estrategias

Control

Copias

Planificación

Continuity: A Perspective and Future Research Outlook. *Applied Sciences*, 13(21), 11846. <https://doi.org/10.3390/app132111846>

Saha, B. (2025). Best Practices for IT Disaster Recovery Planning in Multi-Cloud Environments. *SSRN Electronic Journal*. <https://doi.org/10.2139/ssrn.5224693>

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database system concepts* (Seventh edition). McGraw-Hill Education.

Tatineni, S. (2023). CLOUD-BASED BUSINESS CONTINUITY AND DISASTER RECOVERY STRATEGIES. *International Research Journal of Modernization in Engineering Technology and Science*. <https://doi.org/10.56726/IRJMETs46236>

Thallapally, N. (2025). How to Build a Strong Data Backup and Disaster Recovery Strategy. *Journal of Electrical Systems*, 21(1s), 70-76. <https://doi.org/10.52783/jes.8334>

Zgureanu, A. (2022). The role of RPO and RTO in disaster recovery planning. *30 Years of Economic Reforms in the Republic of Moldova: Economic Progress via Innovation and Competitiveness, Vol III*, 221-232. <https://doi.org/10.53486/9789975155663.26>

Zhao, H., Zhao, Z., Yang, W., Lu, W., Li, H., Du, X., & Key Laboratory of Data Engineering and Knowledge Engineering (Renmin University of China), Ministry of Education, Beijing 100872, China; School of Information, Renmin University of China, Beijing 100872, China. (2022). Experimental Study of Concurrency Control Algorithms in Main-memory Database Systems. *International Journal of Software and Informatics*, 12(1), 55-88. <https://doi.org/10.21655/ijsi.1673-7288.00276>